

Reflog

Rapport de test d'une forge Git collaborative. 51 cas du plan exécutés dans leur intégralité, complétés par 13 tests de cas négatifs et de frontières de permissions.

PRODUIT reflog.dev	COUVERTURE 51 / 51 cas · + 13 complémentaires	ENVIRONNEMENT Chromium 152 · Playwright · git / OpenSSH
COMPTES reflogqa1 · reflogqa2 · reflogqa3	PREUVES 132 captures · results.jsonl	

54
Réussis

6
En échec

4
Bloqués

64
Tests exécutés

01 Synthèse

Les 40 fonctionnalités déclarées « Done » dans le plan passent toutes les tests — sans exception.

Les 11 fonctionnalités restantes sont déclarées « In progress ». Sur ces 11, une seule (F50 — Markdown) fonctionne déjà intégralement ; les 10 autres ne sont pas opérationnelles. Aucun écart n'a été trouvé entre l'état d'avancement annoncé par l'équipe et l'état réel du produit : le statut de développement du plan est fiable.

STATUT DÉCLARÉ	CAS	RÉUSSIS	ÉCHECS	BLOQUÉS
Done	40	40	0	0
In progress	11	1	6	4

Le socle du produit est solide : création de compte et de session, gestion des clés SSH, cycle Git complet (clone / push / pull / branches) en SSH, navigation de code avec coloration syntaxique et diffs, organisations, et le cycle de vie complet des merge requests — création, revue, approbation, blocage, fusion, fermeture — fonctionnent tous conformément aux attendus.

Les diffs et statistiques affichés dans l'interface ont été systématiquement recoupés avec `git show`, `git log` et `git diff` : ils correspondent exactement.

02 Anomalies et points bloquants

BLOQUÉ • F12 • F13 • F14 • F15

Le CLI Reflog est introuvable

La feuille « Info & Access » du plan indique comme point d'accès au CLI le littéral `https://TODO`. Aucun binaire n'est récupérable depuis le produit non plus : `/cli`, `/download`, `/downloads`, `/docs`, `/install`, `/releases` et `/api/cli` renvoient tous **HTTP 404**, et l'application web n'expose aucun lien de téléchargement. Quatre fonctions critiques du plan n'ont donc pas pu être évaluées.

Action attendue : publier le binaire ou les instructions d'installation, et compléter la feuille « Info & Access » — les identifiants des trois comptes de test y sont également restés à `TODO`.

ÉCHEC • F28 • F29 — PRIORITÉ HAUTE

Git par HTTPS est inutilisable par tout client

C'est le constat technique le plus important de la campagne. Le point d'entrée Git smart-HTTP ne se comporte pas comme le protocole l'exige :

```
$ curl -I -u 'reflogqa1:<mot de passe>' \  
  'https://reflog.dev/reflogqa1/qa-repo-alpha.git/info/refs?service=git-upload-pack'  
  
HTTP/2 307  
location: /login?redirect=%2Freflogqa1%2Fqa-repo-alpha.git%2Finfo%2Frefs...
```

Le serveur répond **307 vers une page de login HTML**, au lieu du **401** accompagné de `WWW-Authenticate: Basic` qu'attend Git. Conséquences :

- les identifiants HTTP Basic sont **totalement ignorés** — même fournis explicitement, ils ne sont jamais évalués ;
- aucun credential helper, y compris celui de Reflog (F13), ne peut fonctionner tant que le challenge n'est pas émis ;
- `git clone` échoue sur un message peu explicite : `fatal: unable to update url base from redirection`.

Le HTTPS étant le seul chemin pour les utilisateurs sans clé SSH, cela ferme aujourd'hui tout un mode d'accès.

Action attendue : répondre **401** + `WWW-Authenticate` sur `*.git/info/refs`, `git-upload-pack` et `git-receive-pack`, et ne rediriger vers `/login` que les requêtes de navigateur.

ÉCHEC • F48 • F49 • F51

La revue de code ligne à ligne n'existe pas encore

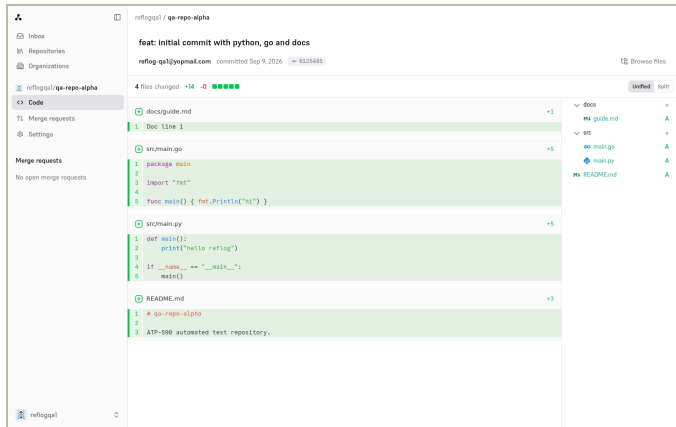
Ces trois fonctions forment un bloc cohérent et manquant. Le diff de l'onglet *Files changed* s'affiche parfaitement — grille `<code>` de `div[data-line]`, coloration syntaxique, bascule Unified / Split — mais reste **purement présentationnel** : le survol d'une ligne, le clic simple et le cliquer-glisser sur une plage de lignes n'ouvrent aucun composeur (zéro `textarea` créé dans les trois cas), et il n'existe aucun bouton de gouttière.

Sans point d'ancrage dans le diff, F51 n'a pas de support, et les commentaires de F49 restent une liste plate sans bouton « Reply » ni imbrication. C'est la seule zone nettement en retrait : la revue **au niveau de la MR** — approuver, demander des modifications, bloquer la fusion — fonctionne parfaitement.

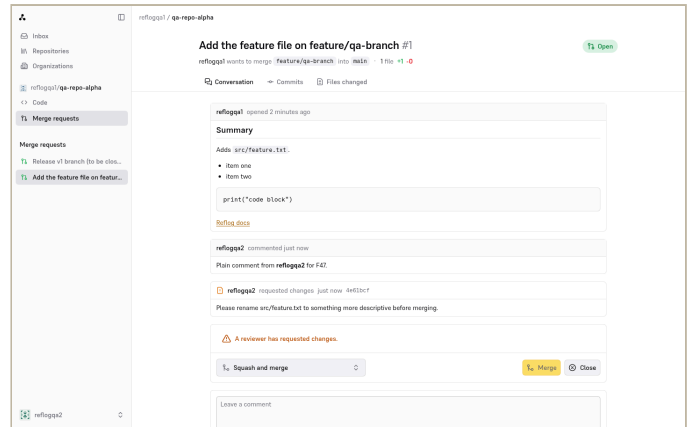
Le range diff est amorcé mais non exposé

Après réécriture d'une branche de MR (commit amendé + un commit ajouté) suivie d'un `git push --force-with-lease`, le fil de conversation **enregistre bien l'événement** : `reflogqa1 pushed 239ce11 → d3c8341`.

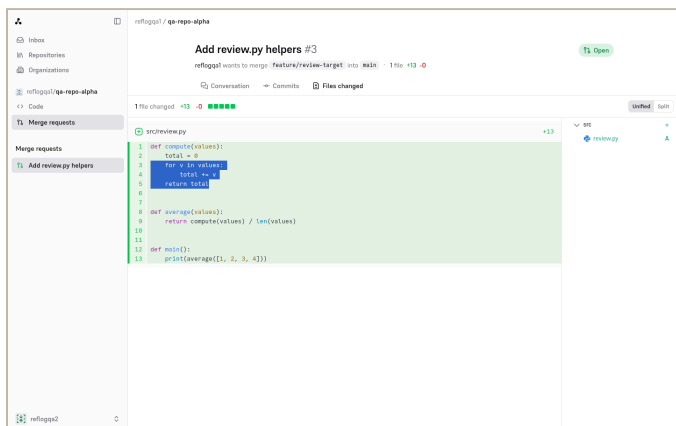
Les deux sommets d'itération sont donc connus du serveur — la donnée nécessaire est déjà là. Il manque uniquement la vue qui les compare : aucun contrôle « range-diff », « compare versions » ou « iteration » n'existe sur les onglets Conversation, Commits ou Files changed, et l'onglet Commits ne liste que l'itération courante.



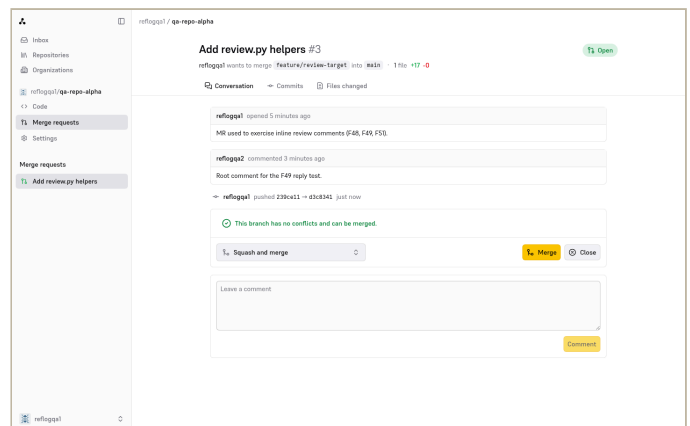
F32 — Le diff du commit `6125485` affiche « 4 files changed +14 -0 » avec le détail par fichier. Correspondance exacte avec `git show --stat`.



F44 — La revue « requested changes » affiche le bandeau bloquant. Le bouton Merge est désactivé (`disabled=true`, opacité 0.6).



F48 — Le diff s'affiche parfaitement mais reste purement présentationnel : survol, clic et cliquer-glisser n'ouvrent aucun compositeur de commentaire.



F35 — Le force-push est bien enregistré (« pushed 239ce11 → d3c8341 ») : les deux itérations sont connues du serveur, mais aucune vue ne les compare.

03 Points forts relevés

Ces constats ne sont pas demandés par le plan, mais ont été vérifiés explicitement et méritent d'être signalés.

- **Le contrôle d'accès est réellement appliqué au niveau Git, pas seulement dans l'interface.** Passer un collaborateur de Write à Read fait immédiatement échouer son `git push` (`ERROR: Insufficient permissions.`) tout en laissant le `fetch` fonctionner ; son retrait fait échouer le clone. La révocation d'une clé SSH prend effet sans délai.
- **Les messages d'erreur ne divulguent pas d'information.** Un clone par un utilisateur retiré renvoie `ERROR: Repository not found.` plutôt que « permission refusée » — l'existence du dépôt privé n'est pas confirmée. Le login répond « Invalid username or password » sans préciser lequel est erroné.
- **Les actions destructrices sont bien protégées.** Supprimer un dépôt ou une organisation exige la saisie manuelle du nom ; les retraits passent par un `alertdialog` explicite.
- **La restriction Ed25519 est effectivement appliquée** — une clé RSA 2048 est rejetée à l'ajout.

- **Le changement de mot de passe déconnecte réellement les autres sessions**, comme annoncé dans le dialogue.
- **L'interface est correctement accessible** : `aria-label` systématiques, `role=treeitem` sur l'arbre de fichiers, `role=alertdialog` sur les confirmations, `aria-pressed` sur les contrôles segmentés — ce qui a d'ailleurs rendu l'automatisation nettement plus fiable.
- **Aucune erreur JavaScript** relevée en console sur l'ensemble des parcours.

04 Observations mineures

- **Attribution des commits par e-mail.** Dans l'historique, l'auteur s'affiche par son adresse git (`reflog-qa1@yopmail.com`) plutôt que par le nom d'affichage du compte, alors que F04 attend que ce nom apparaisse « dans toute l'UI ».
- **Aucune confirmation d'e-mail à l'inscription.** Conforme à l'attendu de F01, mais rien ne prouve aujourd'hui que l'adresse saisie appartient à l'utilisateur.
- **Pas de badge « Private » sur la page du dépôt.** Les dépôts sont bien privés (vérifié en C03) mais la vue n'affiche aucun indicateur de visibilité.

05 Méthode

Chaque cas a été exécuté en suivant à la lettre les étapes du plan. Trois principes ont guidé la campagne.

- **Vérification côté serveur, pas seulement à l'écran.** Chaque fois qu'un cas avait une contrepartie Git, le résultat affiché a été recoupé en ligne de commande : `git show --stat` pour F32, `git log -- <fichier>` pour F34, `git diff base..compare` pour F33, et le comptage des parents du commit de fusion pour prouver que la stratégie « Squash and merge » de F45 a bien été appliquée.
- **Test réel des permissions.** Les cas de rôles (F24, F25, F26) n'ont pas été validés sur le simple affichage : un second compte doté de sa propre clé SSH a tenté clone, fetch et push après chaque changement de rôle.
- **Distinction stricte entre défaut du produit et limite du harnais.** Plusieurs assertions automatiques ont d'abord échoué pour des raisons d'outillage — le visualiseur de code n'expose pas son contenu via `innerText`, et un sélecteur trop large visait le menu de stratégie plutôt que le bouton Merge. Ces cas ont été rejoués et contrôlés visuellement sur capture avant tout verdict ; aucun n'est comptabilisé comme un défaut.

06 Résultats détaillés — cas du plan

ID	FONCTIONNALITÉ	CATÉGORIE	STATUT DÉCLARÉ	VERDICT	CONSTAT
F01	Créer un compte	Critique	Done	PASS	Trois comptes créés à la suite (reflogqa1/2/3). À chaque fois : compte créé, session ouverte immédiatement sans confirmation e-mail , atterrissage sur l'Inbox avec la sidebar Inbox / Repositories / Organizations / Merge requests. La disponibilité du nom d'utilisateur est vérifiée à la frappe ; la case newsletter est bien facultative.
F02	Se connecter	Critique	Done	PASS	Connexion réussie avec le nom d'utilisateur et avec l'e-mail (les deux testés). Redirection vers l'Inbox, sidebar complète et navigable.
F03	Se déconnecter	Critique	Done	PASS	Menu utilisateur en bas de sidebar → « Log out ». Session détruite, redirection vers <code>/login</code> . Une page protégée redemandée ensuite renvoie <code>/login?redirect=%2F</code> : la session est bien invalidée côté serveur.
F04	Changer le nom d'affichage	Interface	Done	PASS	Nom d'affichage changé en « QA Tester Alpha » via le crayon. Répercuté immédiatement dans toute l'UI sans reconnexion . Nom d'utilisateur et e-mail inchangés, conformément à l'attendu.

ID	FONCTIONNALITÉ	CATÉGORIE	STATUT DÉCLARÉ	VERDICT	CONSTAT
F05	Changer le mot de passe	Secondaire	Done	PASS	Dialogue « Change password » avec 3 champs (actuel / nouveau / confirmation) et l'avertissement « This will sign out all other active sessions ». Après changement : le nouveau mot de passe fonctionne, l'ancien est rejeté , et la seconde session ouverte a bien été déconnectée et renvoyée vers <code>/login</code> . Les trois volets de l'attendu sont vérifiés.
F06	Changer la photo de profil	Interface	Done	PASS	Upload d'un PNG 256×256 via « Upload photo ». Le champ n'accepte que <code>image/jpeg</code> , <code>image/png</code> , <code>image/webp</code> , conforme au plan. Avatar mis à jour immédiatement dans l'UI.
F07	Supprimer la photo de profil	Interface	Done	PASS	Option « Remove photo » présente dans le menu de l'avatar. Après clic, retour à l'avatar généré par défaut, immédiatement visible.
F08	Ajouter une clé SSH	Critique	Done	PASS	Clé Ed25519 ajoutée avec nom, type et empreinte affichés : <code>SHA256:uzWmb8iWmQxWep0+/XuZtdcY1v5Rv7oNFhsGfxfS1Ao</code> . Le libellé de type ED25519 est bien affiché. Usage Git réellement exercé en F17/F18.
F09	Renommer une clé SSH	Secondaire	Done	PASS	Renommage via le crayon (dialogue « Rename SSH key »). Nouveau nom affiché, et empreinte identique avant/après (<code>SHA256:uzWmb8...</code> <code>S1Ao</code>) : la clé n'a pas été régénérée.

ID	FONCTIONNALITÉ	CATÉGORIE	STATUT DÉCLARÉ	VERDICT	CONSTAT
F10	Supprimer une clé SSH	Secondaire	Done	PASS	Suppression via la corbeille, avec confirmation explicite « Delete SSH key? "QA key A renamed" will no longer be able to authenticate. » La clé disparaît de la liste (« You haven't added any SSH key yet. »). Rejet Git vérifié séparément (voir F10b).
F11	Révoquer une session	Secondaire	Done	PASS	Deux sessions ouvertes sur le même compte. Confirmation « Revoke session? ... will be signed out ». La ligne disparaît (2 → 1 session révocable) et la session révoquée est effectivement renvoyée vers /login à sa requête suivante.
F12	Authentification CLI	Critique	In progress	BLOQUÉ	Non testable. La feuille « Info & Access » du plan indique comme point d'accès CLI le littéral <code>https://TODO</code> . Aucun binaire n'est récupérable depuis le produit non plus : <code>/cli</code> , <code>/download</code> , <code>/downloads</code> , <code>/docs</code> , <code>/install</code> , <code>/releases</code> , <code>/api/cli</code> renvoient tous HTTP 404 , et l'application web n'expose aucun lien de téléchargement.
F13	Credential helper Git	Critique	In progress	BLOQUÉ	Non testable (dépend de F12). Constat complémentaire obtenu au niveau HTTP : <code>GET /reflogqa1/qa-repo-alpha.git/info/refs?service=git-upload-pack</code> répond HTTP 307 vers <code>/login?redirect=...</code> , une page HTML — au lieu du <code>401</code> + <code>WWW-Authenticate: Basic</code> qu'impose le protocole Git smart-HTTP. Les identifiants HTTP Basic sont totalement ignorés .
F14	Créer une MR depuis le CLI	Critique	In progress	BLOQUÉ	Non testable : aucun binaire CLI disponible (cf. F12).

ID	FONCTIONNALITÉ	CATÉGORIE	STATUT DÉCLARÉ	VERDICT	CONSTAT
F15	Consignes agent via le CLI	Secondaire	In progress	BLOQUÉ	Non testable : aucun binaire CLI disponible (cf. F12).
F16	Créer un dépôt	Critique	Done	PASS	Dépôt créé depuis <code>/new</code> (propriétaire, nom, description). Ouvre directement sur la vue code à <code>/reflogqa1/qa-repo-alpha/tree</code> , apparaît dans Repositories, prêt à cloner. Le formulaire indique « Repositories in Reflog are private » — la confidentialité est confirmée par C03 (clone anonyme refusé).
F17	Cloner un dépôt (SSH)	Critique	Done	PASS	<code>git clone</code> SSH réussi. <code>git log</code> du clone strictement identique à celui du dépôt source (3 commits) : l'historique complet est bien restitué.
F18	Pousser des commits (SSH)	Critique	Done	PASS	3 commits poussés sur <code>main</code> via SSH. Push accepté, commits visibles dans l'UI en quelques secondes (vérifié en F32/F34).
F19	Récupérer les changements (SSH)	Critique	Done	PASS	Commit poussé depuis un second clone, puis <code>git pull</code> : passage de 3 à 4 commits, le nouveau commit <code>c15645b</code> est bien intégré. Cohérent avec l'UI web.
F20	Créer une branche	Secondaire	Done	PASS	<code>git checkout -b feature/qa-branch</code> puis <code>git push -u origin</code> . La branche apparaît dans <code>git ls-remote --heads</code> et est sélectionnable dans le sélecteur de révision de la vue code.
F21	Supprimer une branche	Secondaire	Done	PASS	<code>git push origin --delete tmp/to-delete</code> . La branche disparaît des références distantes et du sélecteur ; les autres branches sont intactes.

ID	FONCTIONNALITÉ	CATÉGORIE	STATUT DÉCLARÉ	VERDICT	CONSTAT
F22	Changer la branche par défaut	Secondaire	Done	PASS	Sélecteur de branche par défaut → <code>release/v1</code> , appliqué immédiatement. La vue code s'ouvre sur cette branche (fichier <code>VERSION</code> , propre à <code>release/v1</code> , présent dans l'arbre). Voir aussi F22b pour le clone.
F23	Changer la stratégie de fusion	Secondaire	Done	PASS	Les 3 stratégies sont proposées (Merge commit / Squash and merge / Rebase and merge). Passage à « Squash and merge » persistant après rechargement, et effectivement pré-sélectionné à la création de la MR suivante (constaté en F41).
F24	Ajouter un collaborateur	Critique	Done	PASS	Dialogue « Add a collaborator » avec les 3 rôles Read / Write / Admin et leur description. <code>reflogqa2</code> ajouté en Write, listé aussitôt. L'accès réel accordé par le rôle est vérifié en F24b.
F25	Changer le rôle d'un collaborateur	Secondaire	Done	PASS	Rôle rétrogradé Write → Read via le contrôle segmenté, effet immédiat. Le push du collaborateur est alors refusé — <code>ERROR: Insufficient permissions. Contact repository owner for more information.</code> — tandis que le <code>fetch</code> (lecture) continue de fonctionner. Séparation lecture/écriture correcte.
F26	Retirer un collaborateur	Secondaire	Done	PASS	Confirmation « Remove collaborator? <code>reflogqa2</code> will lose access to this repository. » Le collaborateur disparaît de la liste et son clone SSH est ensuite refusé avec <code>ERROR: Repository not found.</code> — bonne pratique : le message ne divulgue pas l'existence du dépôt.

ID	FONCTIONNALITÉ	CATÉGORIE	STATUT DÉCLARÉ	VERDICT	CONSTAT
F27	Supprimer un dépôt	Secondaire	Done	PASS	Suppression exigeant la saisie du nom du dépôt en confirmation. Retour à <code>/repositories</code> , dépôt absent de la liste, et son URL ne résout plus.
F28	Clone / pull authentifié HTTPS	Critique	In progress	ÉCHEC	Échec. La moitié « non authentifié » de l'attendu est correcte (clone anonyme refusé, cf. C03). Mais aucun clone HTTPS authentifié n'est possible : le helper de credentials (F13) est indisponible, et le point d'entrée Git HTTPS n'émet jamais de challenge d'authentification — il redirige systématiquement (307) vers la page de login HTML. Testé avec identifiants dans l'URL : <code>fatal: unable to update url base from redirection</code> .
F29	Pousser des commits en HTTPS	Critique	In progress	ÉCHEC	Échec , même cause que F28 : aucun clone HTTPS authentifié ne peut être obtenu, donc aucun push HTTPS n'est réalisable. Le point d'entrée <code>git-receive-pack</code> se comporte à l'identique (307 vers <code>/login</code>).
F30	Parcourir les fichiers	Interface	Done	PASS	Arbre racine <code>docs / src / README.md</code> conforme. <code>src/</code> se déploie sur <code>main.go</code> + <code>main.py</code> , <code>docs/</code> sur <code>guide.md</code> . Le sélecteur de révision liste bien <code>main</code> , <code>release/v1</code> et <code>feature/qa-branch</code> . Le changement de branche est vérifié en F30b.

ID	FONCTIONNALITÉ	CATÉGORIE	STATUT DÉCLARÉ	VERDICT	CONSTAT
F31	Coloration syntaxique	Interface	Done	PASS	<code>src/main.py</code> et <code>src/main.go</code> s'affichent intégralement, avec numéros de ligne et coloration syntaxique correcte par langage (22 spans de tokens, 8 couleurs distinctes : mots-clés, chaînes, nombres différenciés).
F32	Diff d'un commit	Interface	Done	PASS	Commit <code>6125485</code> : l'UI affiche « 4 files changed +14 -0 » avec le détail par fichier (<code>docs/guide.md</code> +1, <code>src/main.go</code> +5, <code>src/main.py</code> +5, <code>README.md</code> +3) et un bascule Unified/Split. Correspondance exacte avec <code>git show --stat</code> (4 fichiers, 14 insertions).
F33	Comparer deux références	Interface	Done	PASS	Vue de comparaison à <code>/compare/main...feature%2Fqa-branch</code>, affichant « 1 file changed » — conforme à <code>git diff main..feature/qa-branch</code> (1 fichier, <code>src/feature.txt</code>, +1).
F34	Historique d'un fichier	Interface	Done	PASS	Historique de <code>src/main.py</code> à <code>/commits/main/src/main.py</code> : les 2 commits touchant ce fichier sont listés, et les commits ne touchant que <code>docs/</code> sont correctement exclus. Identique à <code>git log -- src/main.py</code>.

ID	FONCTIONNALITÉ	CATÉGORIE	STATUT DÉCLARÉ	VERDICT	CONSTAT
F35	Range diff	Secondaire	In progress	ÉCHEC	Échec. Après réécriture de la branche (commit amendé + 1 commit ajouté) et <code>git push -force-with-lease</code>, le fil enregistre bien l'événement — « reflogqa1 pushed 239ce11 → d3c8341 » — donc les deux itérations sont connues du serveur. Mais aucune vue de range-diff / comparaison de versions n'est exposée : aucun contrôle « range-diff », « iteration », « version » ou « compare » sur les onglets Conversation, Commits ou Files changed, et l'onglet Commits ne liste que l'itération courante sans moyen de sélectionner la précédente.
F36	Créer une organisation	Critique	Done	PASS	Organisation <code>qa-org-atp590</code> créée (nom 3–30 caractères : lettres, chiffres, tirets ; nom d'affichage optionnel). Listée sur <code>/organizations</code>. Le rôle du créateur est vérifié en F36b.
F37	Supprimer une organisation	Secondaire	Done	PASS	Suppression exigeant la saisie du nom de l'organisation. Elle disparaît de <code>/organizations</code> (« You are not a member of any organization yet ») et son URL de settings ne résout plus vers l'organisation — elle redirige proprement vers la liste plutôt que d'afficher une 404.
F38	Ajouter un membre	Critique	Done	PASS	Dialogue « Add a member » proposant les 4 rôles Read / Write / Admin / Owner. <code>reflogqa2</code> ajouté et présent dans la liste des membres immédiatement.

ID	FONCTIONNALITÉ	CATÉGORIE	STATUT DÉCLARÉ	VERDICT	CONSTAT
F39	Retirer un membre	Secondaire	Done	PASS	Confirmation « Remove member? reflogqa2 will lose access to this organization. » Le membre disparaît de la liste.
F40	Changer le rôle d'un membre	Secondaire	Done	PASS	Contrôle segmenté Read / Write / Admin / Owner à côté du membre. Passage Write → Admin appliqué immédiatement et persistant après rechargement (<code>aria-pressed="true"</code> sur Admin, confirmé dans le DOM).
F41	Créer une merge request	Critique	Done	PASS	MR n°1 créée <code>feature/qa-branch</code> → <code>main</code> . La stratégie de fusion est pré-remplie avec le défaut du dépôt (« Squash and merge »), ce qui valide aussi l'attendu de F23. Un onglet Diff permet de prévisualiser le changement avant création. Ouvre sur l'onglet Conversation.
F42	Consulter une merge request	Critique	Done	PASS	Les 3 onglets Conversation / Commits / Files changed sont présents. En-tête : « reflogqa1 wants to merge <code>feature/qa-branch</code> into <code>main</code> · 1 file +1 -0 ». Files changed affiche <code>src/feature.txt +1</code> avec la ligne ajoutée et le marqueur « A » (ajouté) dans l'arbre. Conforme à git.
F43	Approuver une merge request	Critique	Done	PASS	Revue « approved » par <code>reflogqa2</code> visible dans le fil, le bandeau bloquant est levé et le bouton Merge redevient actif.

ID	FONCTIONNALITÉ	CATÉGORIE	STATUT DÉCLARÉ	VERDICT	CONSTAT
F44	Demander des modifications	Critique	Done	PASS	Revue « requested changes » dans le fil, bandeau « A reviewer has requested changes. » dans le panneau de fusion, et le bouton Merge est bien désactivé (<code>disabled=true</code> , <code>aria-disabled=true</code> , opacité 0.6). Conforme à l'attendu dans ses trois volets.
F45	Fusionner une merge request	Critique	Done	PASS	Fusion en « Squash and merge » avec dialogue « Merge this request? The squash and merge will advance main » et message de commit optionnel. État passé à « Merged ». Vérification git en F45b.
F46	Fermer une merge request	Secondaire	Done	PASS	MR n°2 fermée sans fusion. État « Closed », toujours visible sous le filtre « Closed ». Branche cible intacte (vérifié en F46b).
F47	Commenter une merge request	Critique	Done	PASS	Commentaire posté depuis un second compte, enregistré, attribué à son auteur avec horodatage et visible dans le fil pour les autres membres.
F48	Commentaire de code en ligne	Critique	In progress	ÉCHEC	Non implémenté. Le diff de l'onglet Files changed s'affiche correctement (13 lignes en <code>div[data-line]</code> dans une grille <code><code></code> , avec coloration syntaxique) mais ne porte aucun affordance de commentaire : le survol d'une ligne, le clic simple et le cliquer-glisser sur une plage de lignes n'ouvrent aucun composeur (0 textarea créé dans les trois cas), et il n'existe aucun bouton de gouttière dans le diff. Seuls contrôles présents : Unified / Split et l'arbre de fichiers.

ID	FONCTIONNALITÉ	CATÉGORIE	STATUT DÉCLARÉ	VERDICT	CONSTAT
F49	Répondre à un commentaire	Secondaire	In progress	ÉCHEC	Non implémenté. Le commentaire posté s'affiche comme une entrée de fil plate , sans contrôle « Reply » ni imbrication de thread. Le seul composeur de la page est celui de premier niveau (boutons disponibles : Comment / Approve / Request changes).
F50	Markdown dans les commentaires	Interface	In progress	PASS	Fonctionne déjà bien que déclaré « In progress ». Le titre <code>## Summary</code> est rendu en <code><h*></code> , les 2 puces en <code></code> , le bloc de code en <code><pre></code> , et le lien en <code></code> . Rendu identique entre description de MR et commentaires.
F51	Demander une modif. sur un commentaire	Secondaire	In progress	ÉCHEC	Non implémenté. En excluant le bouton de revue global « Request changes » de la MR, aucun contrôle par commentaire ne permet de marquer un commentaire individuel comme exigeant une modification (0 trouvé), et les entrées de commentaire n'exposent aucun menu de débordement ni action « resolve ». F48 étant absent, la fonctionnalité n'a de toute façon aucun point d'ancrage dans le diff.

07 Tests complémentaires

Ces 13 tests ne figurent pas au plan. Ils ont été ajoutés pour vérifier les cas négatifs, les frontières de permissions et la cohérence entre l'interface et l'état réel du dépôt Git.

ID	TEST	VERDICT	CONSTAT
C01	Le login rejette un mauvais mot de passe	PASS	Mot de passe erroné → maintien sur <code>/login</code> avec le message « Invalid username or password », qui ne révèle pas lequel des deux est incorrect. Bonne pratique.

ID	TEST	VERDICT	CONSTAT
C02	Une clé non-Ed25519 (RSA) est rejetée	PASS	Clé RSA 2048 soumise → rejetée , non ajoutée à la liste. L'interface précise « Paste your Ed25519 public key ». La restriction annoncée par le plan est bien appliquée.
C03	Clone HTTPS anonyme d'un dépôt privé refusé	PASS	<code>git clone</code> HTTPS anonyme d'un dépôt privé → refusé (redirection vers <code>/login</code> , aucun contenu servi). La confidentialité par défaut des dépôts est confirmée.
C04	Clone SSH avec clé non autorisée refusé	PASS	Clone SSH avec une clé non enregistrée → <code>git@reflog.dev: Permission denied (publickey)</code> . Aucun accès accordé.
C05	« Request changes » exige un commentaire	PASS	Avec un composeur vide, le bouton « Request changes » est désactivé (<code>disabled=true</code> , <code>aria-disabled=true</code>), ce qui applique correctement la règle « commentaire obligatoire » énoncée par le plan.
F02b	Connexion par adresse e-mail	PASS	Connexion par adresse e-mail au lieu du nom d'utilisateur : fonctionne également. Le champ accepte bien les deux formes.
F10b	Git refusé après révocation de la clé SSH	PASS	Immédiatement après suppression de la clé dans Settings, un clone authentifié par cette clé échoue : <code>git@reflog.dev: Permission denied (publickey)</code> . La révocation est effective sans délai .
F22b	Clone neuf sur la nouvelle branche par défaut	PASS	Après bascule du défaut sur <code>release/v1</code> , un nouveau <code>git clone</code> SSH place bien la copie de travail sur <code>release/v1</code> (le fichier <code>VERSION</code> , propre à cette branche, est présent). L'attendu « a fresh git clone checks out that branch » est vérifié.
F24b	Collaborateur Write : clone ET push	PASS	<code>reflogqa2</code> , collaborateur Write , clone le dépôt privé en SSH puis pousse un commit sur <code>main</code> avec succès. Le rôle accorde réellement les droits annoncés.
F30b	L'arbre reflète la branche sélectionnée	PASS	Vue de l'arbre sur <code>feature/qa-branch</code> : <code>src/feature.txt</code> apparaît, alors que ce fichier n'existe que sur cette branche. L'arbre reflète bien la révision sélectionnée.
F36b	Le créateur détient le rôle Owner	PASS	La liste <code>/organizations</code> affiche <code>qa-org-atp590</code> avec le badge de rôle « Owner » pour son créateur, et la page de settings liste « reflogqa1 (you) — Full control, including profile and deletion — Owner ».

ID	TEST	VERDICT	CONSTAT
F45b	Fusion vérifiée dans git (squash appliqué)	PASS	Vérification git après fusion : <code>origin/main</code> contient bien <code>src/feature.txt</code> , et le nouveau sommet n'a qu'un seul parent — ce qui confirme que la stratégie « Squash and merge » a réellement été appliquée (pas de commit de fusion à deux parents).
F46b	Fermer une MR laisse la cible intacte	PASS	MR n°2 (<code>release/v1</code> → <code>main</code>) fermée et non fusionnée : le fichier <code>VERSION</code> , propre à <code>release/v1</code> , est absent de <code>origin/main</code> . La branche cible est donc bien restée intacte.

08 Recommandations

PRIORITÉ	ACTION	CAS
Haute	Émettre <code>401</code> + <code>WWW-Authenticate: Basic</code> sur les routes Git smart-HTTP au lieu de rediriger vers <code>/login</code> — cela débloque tout l'accès HTTPS	F28, F29, F13
Haute	Publier le binaire du CLI et compléter la feuille « Info & Access » (CLI et identifiants restés à <code>TODO</code>)	F12 → F15
Moyenne	Implémenter les commentaires de diff ligne à ligne, puis les fils de réponse et le marquage par commentaire	F48, F49, F51
Moyenne	Exposer la vue de range diff — l'événement de force-push est déjà enregistré, la donnée existe	F35
Basse	Rattacher l'e-mail de committer au compte pour afficher le nom d'affichage dans l'historique	F04, F32, F34
Basse	Ajouter un indicateur de visibilité (« Private ») sur la page du dépôt	F16

09 Conclusion

Reflog est un produit dont la partie livrée est **fiable et cohérente** : sur les 40 fonctionnalités annoncées comme terminées, les 40 se comportent conformément à leur résultat attendu — y compris sur les points les plus exigeants : l'application effective des rôles au niveau du protocole Git, la fidélité des diffs par rapport à git, et le blocage de fusion sur revue négative.

Les manques se concentrent sur deux chantiers clairement identifiés et déjà signalés comme en cours par l'équipe : **l'accès HTTPS / CLI** et **la revue de code ligne à ligne**. Le premier est le plus urgent — il ne relève pas d'une fonctionnalité à écrire mais d'un comportement HTTP à corriger, et il conditionne à lui seul quatre fonctions critiques.

La correspondance parfaite entre les statuts déclarés dans le plan et l'état réel du produit mérite d'être soulignée : c'est un indicateur de rigueur de l'équipe, et cela rend le plan exploitable tel quel pour le suivi.